PURBANCHAL UNIVERSITY

KHWOPA ENGINEERING COLLEGE
LIBALI-08, BHAKTAPUR

NEPALI PAPER CURRENCY RECOGNITION USING DEEP LEARNING

Presented by:
Anjali Dhami (750406)
Chandrika Deo (750407)
Dristi Gautam (750410)
Min Maya Tamang (750415)

Supervisor:
Er. Rabindra Phoju

INTRODUCTION

- Object Detection is a computer vision technique that involves identifying objects in images or videos, and localizing them by drawing bounding boxes around them.
- Objects can be text in a document, license plate on a vehicle, face of a person, sign in a sign language and so on.

INTRODUCTION

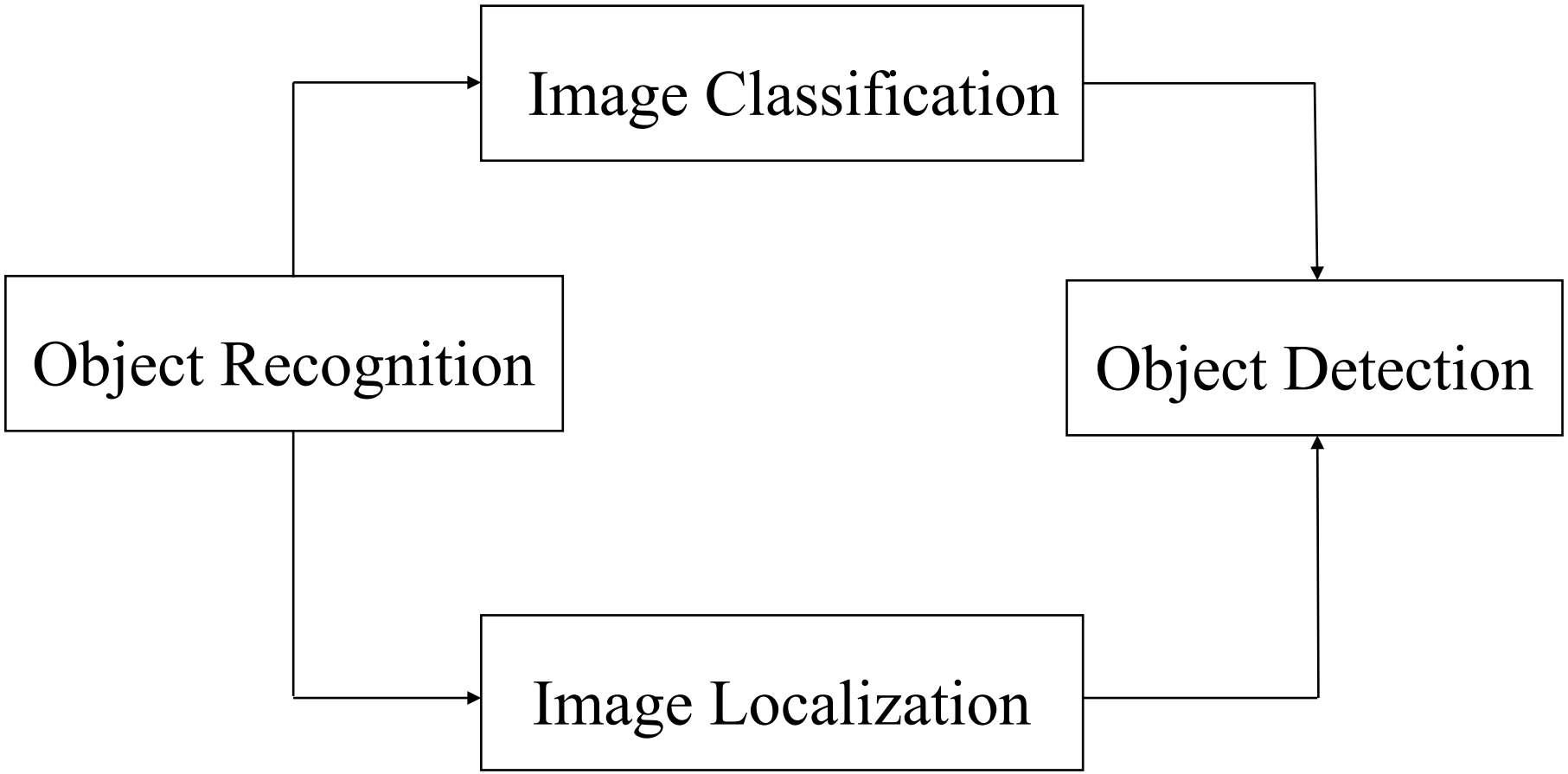


Figure 1: Object detection

INTRODUCTION

- Paper Currency Recognition (PCR) refers to the field of recognizing patterns through image processing techniques which is a type of artificial intelligence.
- It is a system consisting of modules: image acquisition, preprocessing including noise removal, feature extraction, classification and recognition.

INTRODUCTION

- In this project, YOLO technique is used for object detection.
- YOLO is a clever Convolutional Neural Networks (CNN) to detect objects in real-time.
- It requires only a single forward propagation through a neural network to detect objects i.e. prediction in the entire image is done in a single algorithm run.

INTRODUCTION

- YOLOv7 is the fastest and most accurate real-time object detection model for computer vision tasks released in 2022.
- YOLOv7 surpasses all known object detectors in both speed and accuracy in the range from 5 FPS to 160 FPS and has the highest accuracy 56.8% AP among all known real-time object detectors with 30 FPS or higher on GPU V100.

INTRODUCTION

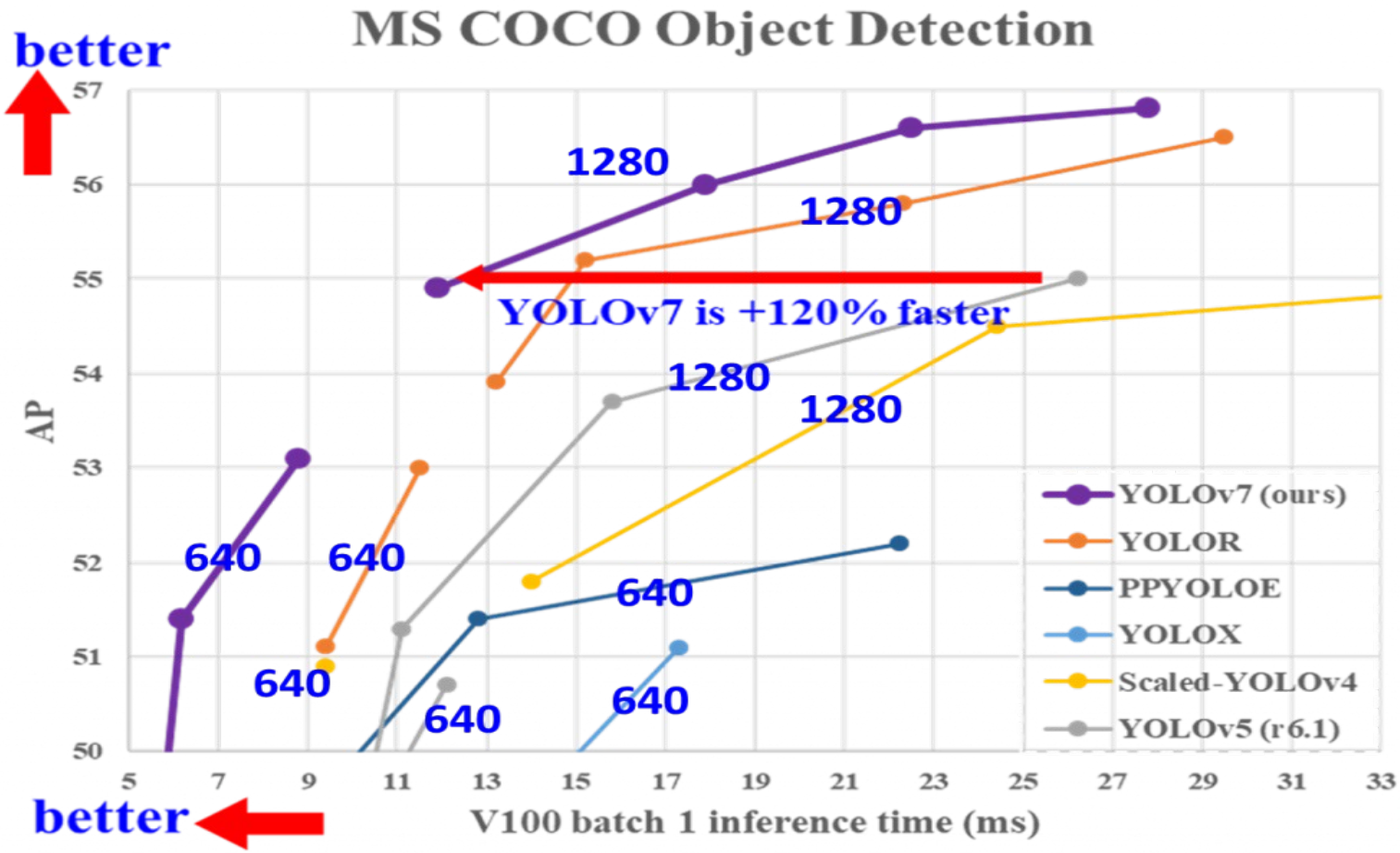


Figure 2: Comparison with other real- time object detectors

(*Source:* https://blog.roboflow.com/yolov7-breakdown)

INTRODUCTION



Figure 3: Mount Everest Bank Note Series Of Nepal Issued Between 2012-2019

INTRODUCTION

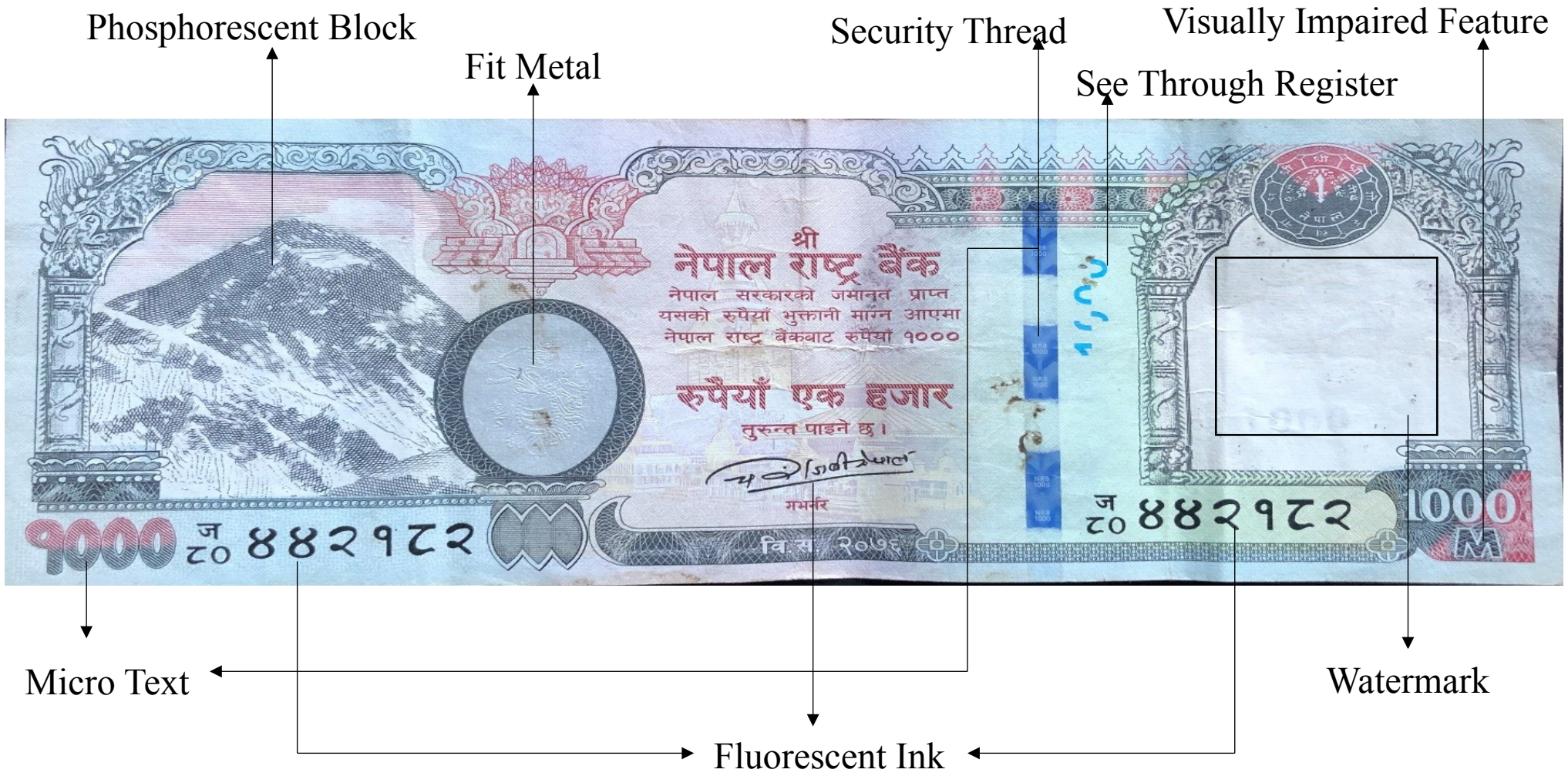


Figure 4: Different Security Feature in Banknotes

PROBLEM STATEMENT

- A system for the recognition of paper currency is very important need for the automation systems in the modern world of today.
- Currency recognition system is implemented to reduce human power to automatically recognize the amount monetary value of currency and perform the monetary transactions without human supervision.
- Automatic identification of fake note is very crucial in some areas like banking, jewelry shops nowadays.

OBJECTIVES

Main Objective

- To implement Nepali paper currency detection using deep learning.

Specific Objective

- To recognize Nepali paper currency using YOLOv7.
- To detect the fake currency.

SCOPE OF STUDY

- Paper currency recognition has various potential applications including banking system, currency monitoring system, money exchange machines, automatic selling-goods system, etc.
- The currency recognition system can be improved by adding the fake currency detection mechanism for counterfeit detection.

LIMITATIONS OF STUDY

- This system is not applicable for coins.
- This system is only applicable for Nepalese currency.
- It cannot recognize worn and torn banknotes.

METHODOLOGY

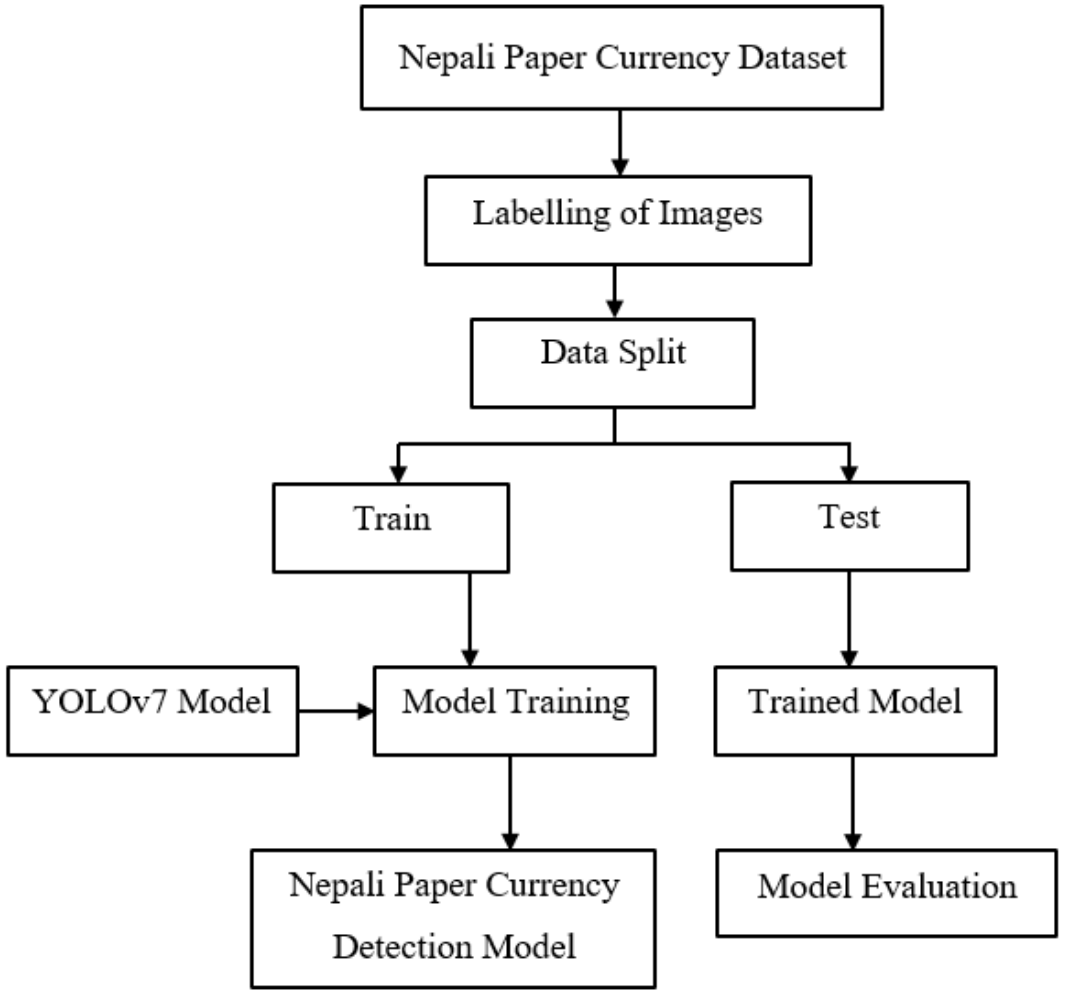


Figure 5: Block Diagram of Nepali Paper Currency Recognition Using Deep Learning

METHODOLOGY

Working Mechanism

- Primarily, 10,000 images of the seven denominations of Nepali Paper Currency is taken, along with the fake ones are then resized into equal size of 640x640 each.
- The resized images are then separated into 8 classes, namely, five, ten, twenty, fifty, hundred, fivehundered, thousand and fake, containing 1250 images each.

METHODOLOGY

- All the images are then labeled in YOLO format using Roboflow and the labeled images are then pre-processed.
- The pre-processed data is split into a 70%, 20% and 10% i.e. into train, valid and test respectively.
- The model training of the split datasets is performed by using by using YOLOv7-tiny algorithm by using GPU to train our custom dataset.
- The model is tested after evaluating the metrics of model's accuracy such as precision, recall and mAP.

METHODOLOGY

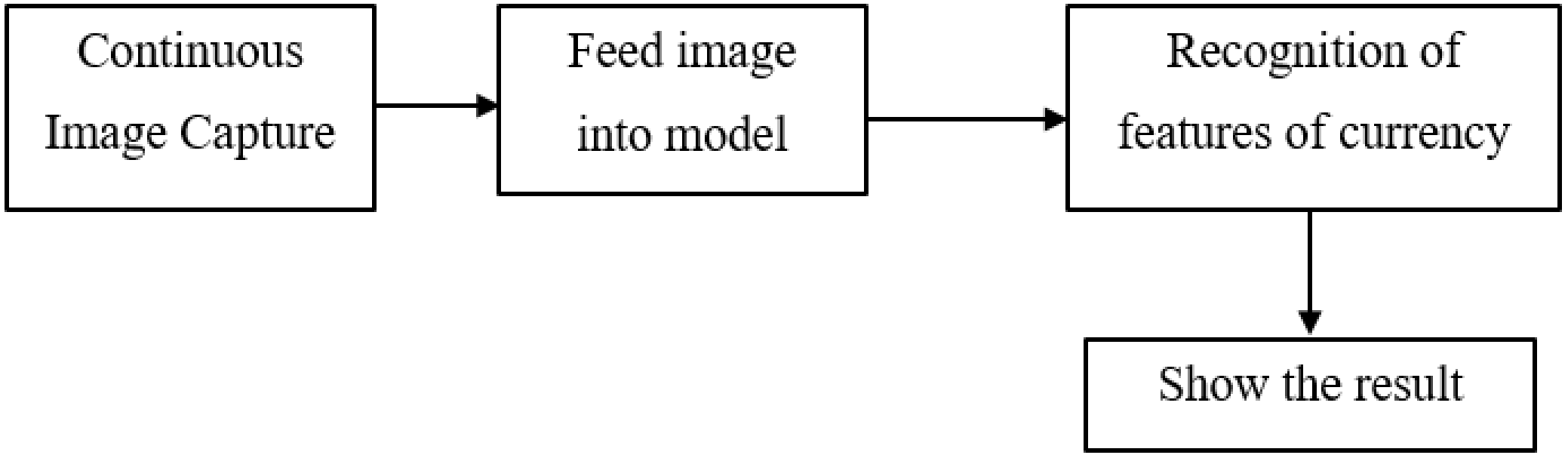


Figure 6: Nepali Paper Currency Recognition Implementation on Jetson Nano

METHODOLOGY

- The SD card image consisting of Jetpack SDK is flashed into the microSD card, which is then inserted into the Jetson Nano board.
- The Jetson Nano board is then connected to a display using HDMI cable along with mouse, keyboard and web camera.
- DC power supply is connected to the J25 Power Jack and the developer kit is turned on automatically.
- The model is implemented into the board and the web camera captures the continuous image and is tested in real time.

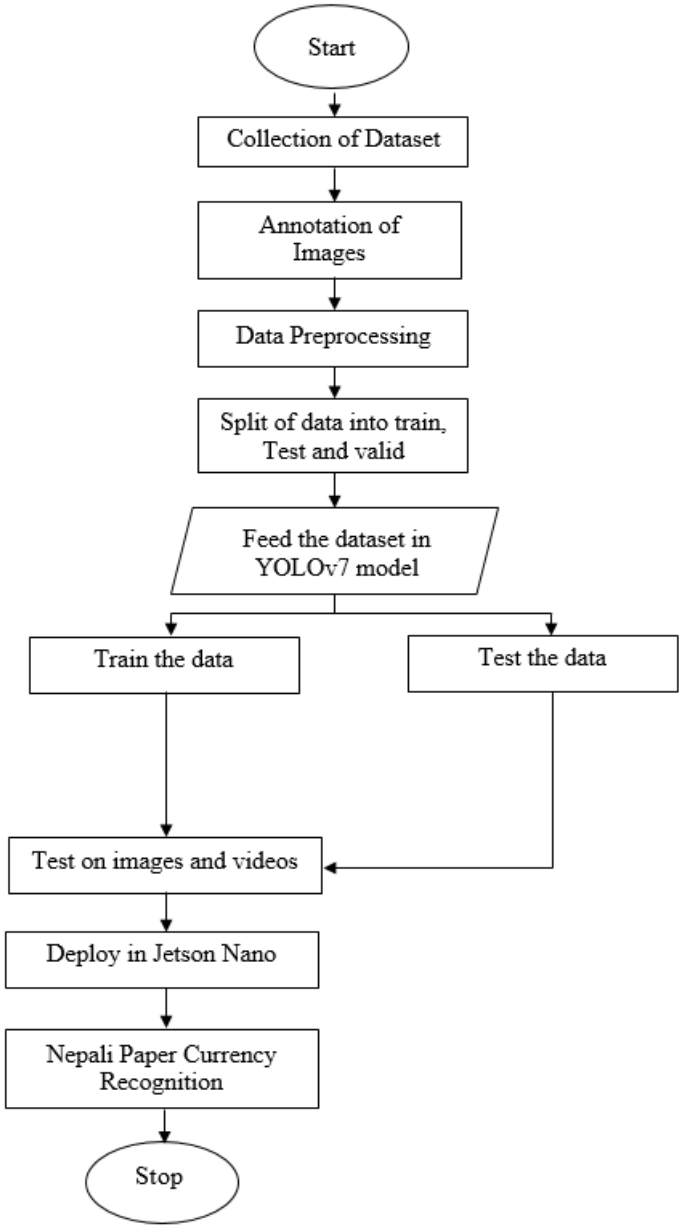


Figure 7: Flowchart of Nepali Paper Currency Recognition Using Deep Learning

METHODOLOGY

YOLOv7 Structure

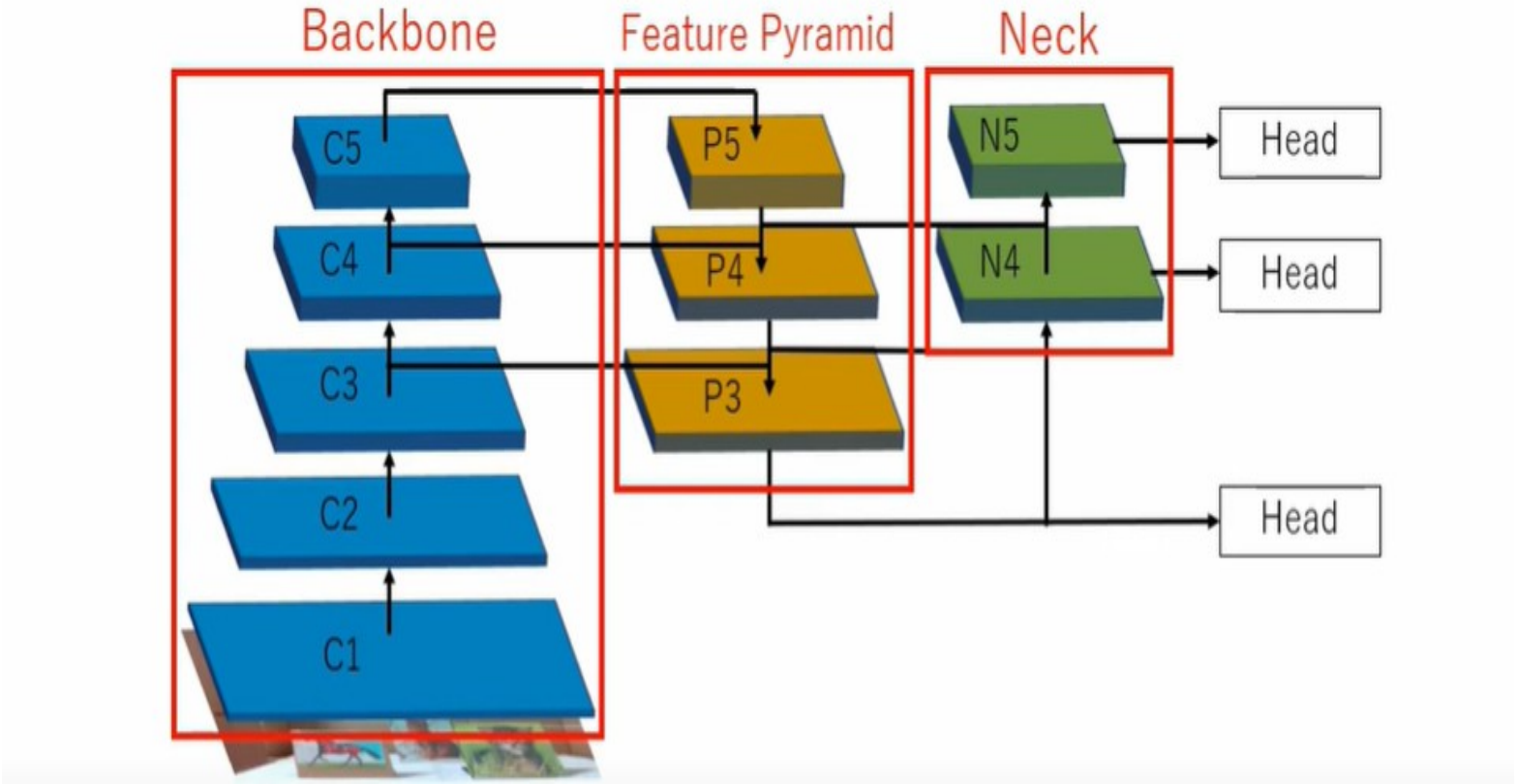


Figure 9: YOLOv7 Structure

METHODOLOGY

YOLOv7 Structure

- Backbone is a CNN (Convolutional Neural Network) that pools images to form features at different granularities.
- Feature pyramids are a basic component in recognition systems for detecting objects at different scales.
- Neck combines and mixes the convolutional network layer representations before passing it to the prediction head.
- Head is a part of network where YOLO predicts the bounding box's locations and the classes of bounding boxes.

METHODOLOGY

Confusion Matrix

- Displays the true positive, true negative, false positive and false negative predictions :

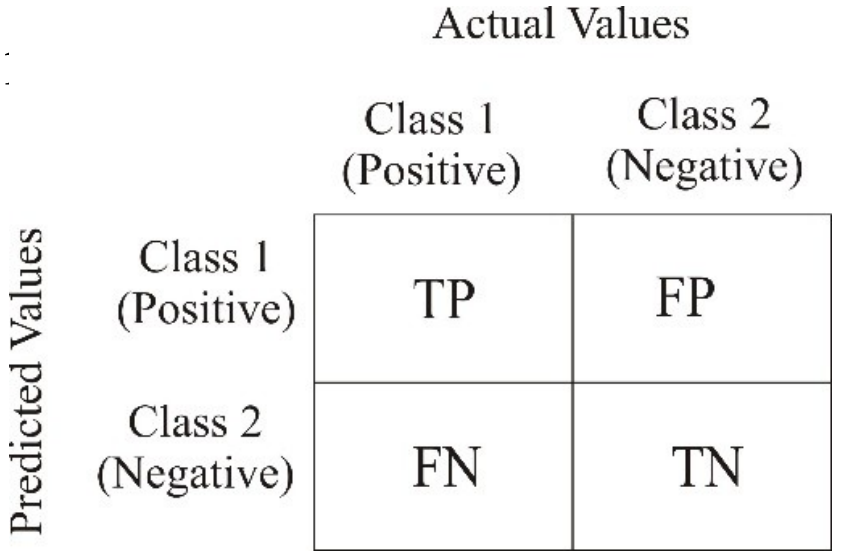


Figure 11: Confusion Matrix

METHODOLOGY

True Positive (TP): The model correctly predicts that the sample belongs to the positive class.

False Positive (FP): The model incorrectly predicts that a sample belongs to a positive class, when it actually belongs to negative class.

False Negative (FN): The model incorrectly predicts that the sample belongs to negative class, when it actually belongs to the positive class.

True Negative (TN): The model correctly predicts that the sample belongs to the negative class.

TOOLS USED

There are two types of tools used in our project:

A. Hardware tools

B. Software tools

A. HARDWARE TOOLS

NVIDIA Jetson Nano Developer Kit

- Small, powerful computer that lets you run multiple neural networks in parallel.
- Used for application like image classification, object detection, segmentation and speech processing.
- We implement and deploy our Nepali Paper Currency Recognition Model in NVIDIA Jetson Nano Developer Kit after being model trained in Google Colab and acceptable accuracy.

A. HARDWARE TOOLS



Figure 12: NVIDIA Jetson Nano Developer Kit

(Source: https://developer.nvidia.com/embedded/jetson-nano-developer-kit)

A. HARDWARE TOOLS

Web camera

- Digital camera connected to computer or other device that generates images and streams live video in real time.
- Typically connected via a USB port or an internet.
- We used web camera for capturing real time video and feed to Jetson Nano Board for the recognition of Nepali paper currency.

A. HARDWARE TOOLS



Figure 13: Web camera

B. SOFTWARE TOOLS

Roboflow

- Framework for better data collection to preprocessing, and model training techniques.
- The custom dataset is uploaded into the roboflow, which is annotated and then labeled.
- Collaboration is possible using the team-sharing feature.
- The pre-processing is then done after the images are annotated and then split into the train, valid and test sets.

B. SOFTWARE TOOLS

Python

- Python is an interpreted, object-oriented, high-level programming language.
- It can also be used as a scripting or glue language to connect existing components together.
- We used Python as programming language for Nepali paper currency recognition program.

B. SOFTWARE TOOLS

Google Colaboratory

- Google Colaboratory is an environment consisting of many pre-installed libraries.
- It is one of the cloud services that support GPU for free and importing a dataset and training models in the data.
- In our project, the pre-processed datasets are implemented into the YOLOv7 model in the Google Colaboratory. The dataset is then trained in the YOLOv7-tiny algorithm in Google Colab.

B. SOFTWARE TOOLS

OpenCV (Open Source Computer Vision)

- Software library of programming functions which provides a wide range of algorithms for image and video analysis, as well as for machine learning and computer vision tasks.
- OpenCV can be applied in the areas which include object detection and recognition, face recognition, gesture recognition, motion analysis and image and video processing.
- OpenCV was used in our project for image I/O and processing tasks.

B. SOFTWARE TOOLS

Anaconda

- An open source distribution of Python programming languages for data science and machine learning which allows us to create isolated environments for our projects.
- For our project, Anaconda provided a distribution of Python that comes pre-installed with many of the packages .
- It also provided a package manager called conda which was used to install additional packages.

RESULTS

- “ In deep learning, data is the fuel and the model is the engine.”

  -Andrew Ng, Computer Scientist

- Quality dataset is the fundamental requirements for training the model.

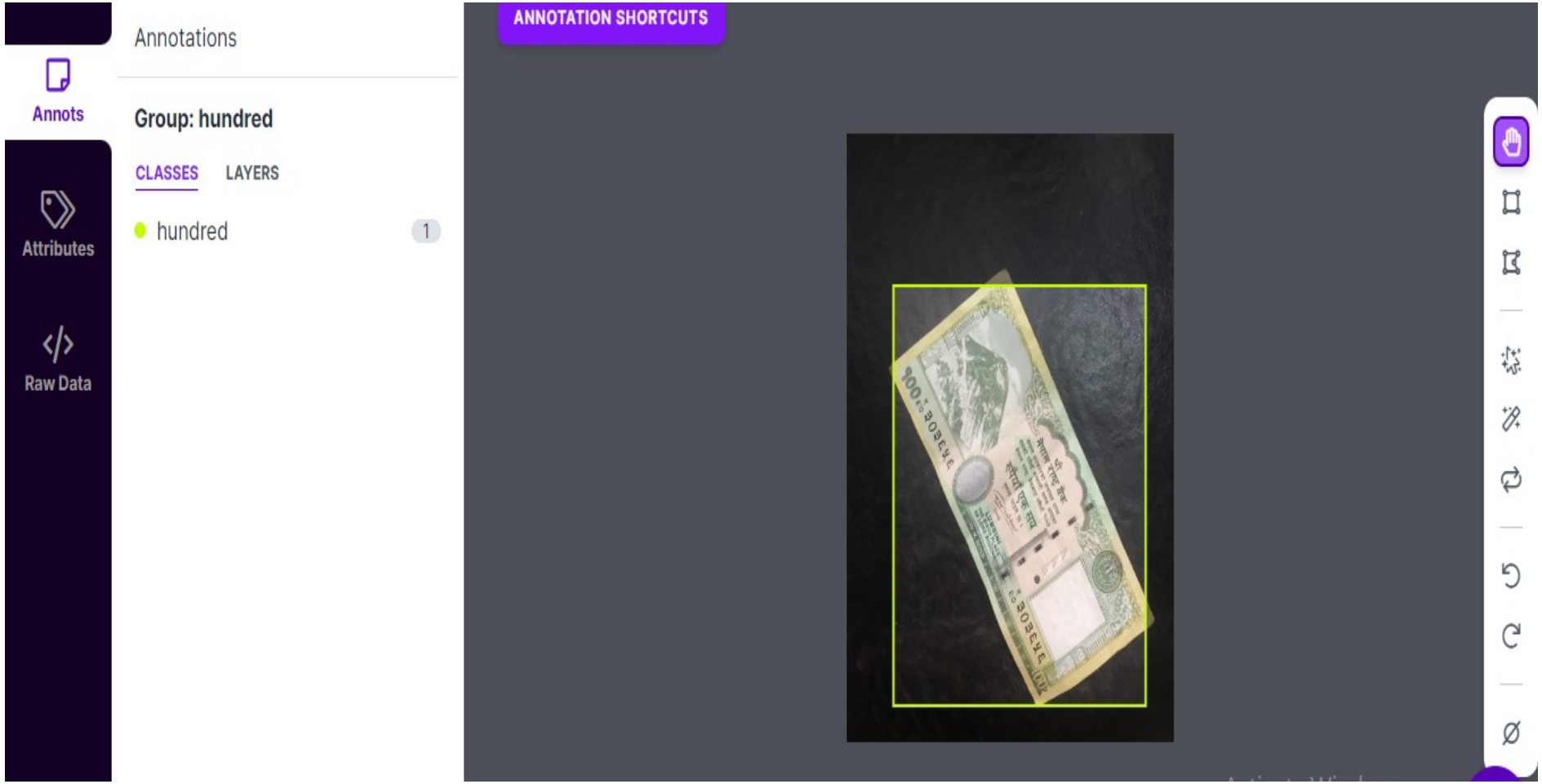


Figure 14: Annotation of Images using Roboflow

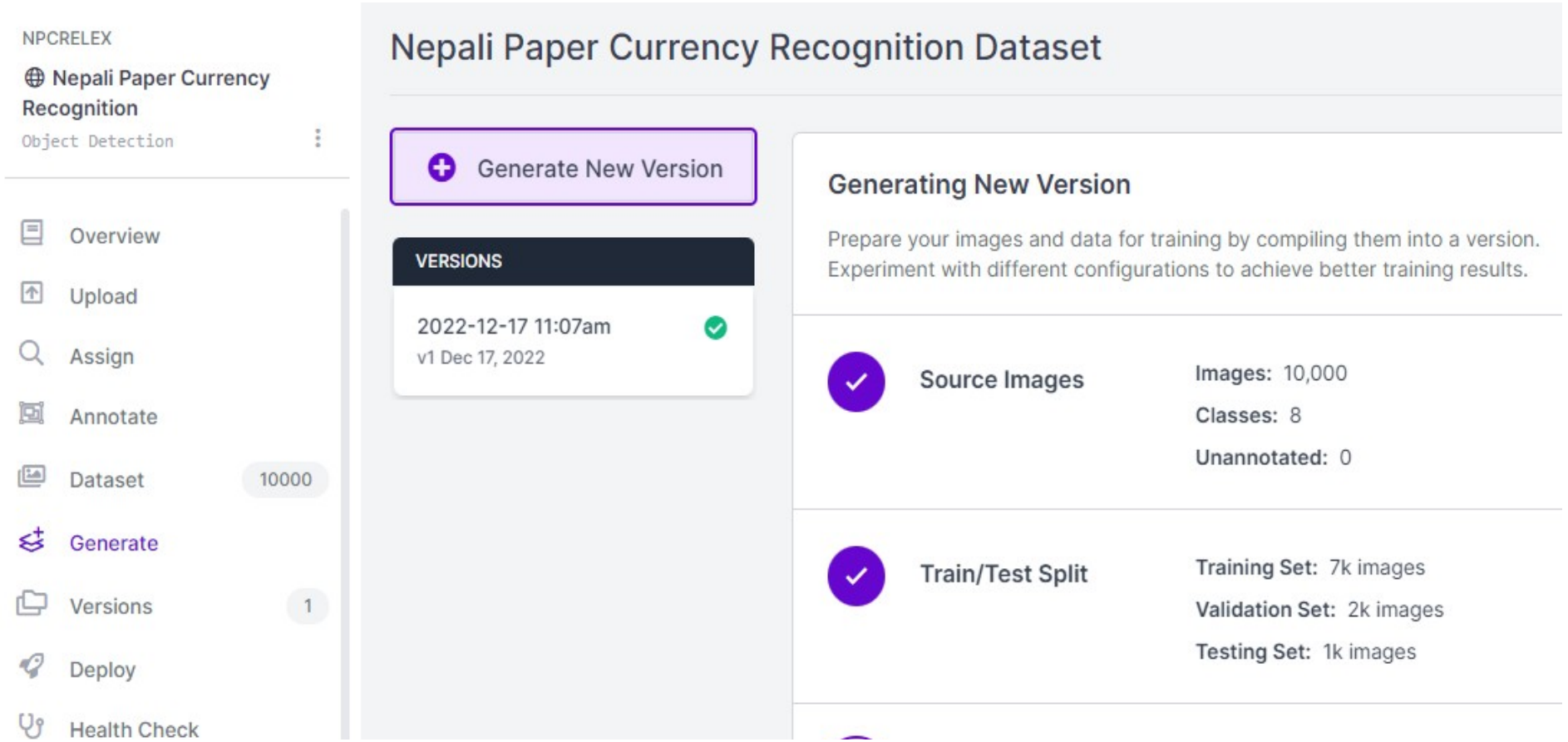


Figure 15: Splitting Dataset into Training, Validation and Testing Set

File Edit Format View Help

```
train: ./data/train
val: ./data/val
test: ./data/test
# number of classes
nc: 8

# class names
names: ['fake', 'fifty', 'five', 'fivehundred', 'hundred', 'ten', 'thousand', 'twenty',]
```

Figure 16: Creating YAML file

```
fake          0 0.5 0.5 1 1
fifty         1 0.32890625 0.45859375 0.36875 0.86171875
five          2 0.36953125 0.5046875 0.6203125 0.990625
fivehundred   3 0.5 0.4984375 1 0.68359375
hundred       4 0.45390625 0.5078125 0.753125 0.92734375
ten           5 0.5109375 0.48125 0.97734375 0.96171875
thousand      6 0.5 0.4640625 1 0.3515625
twenty        7 0.50234375 0.5 0.9953125 1
```

Figure 17: Classes and Labeled Information

```
YOLOR 🚀 v0.1-121-g2fdc7f1 torch 1.13.1+cu116 CUDA:0 (Tesla T4, 15109.75MB)

Namespace(adam=False, artifact_alias='latest', batch_size=64, bbox_interval=-1, bucket='', cache_images=False, cfg='cfg/training/yolov7_custom.yaml', data='data
tensorboard: Start with 'tensorboard --logdir runs/train', view at http://localhost:6006/
hyperparameters: lr0=0.01, lrf=0.1, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1, box=0.05, cls=0.3, cls_pw=
wandb: Install Weights & Biases for YOLOR logging with 'pip install wandb' (recommended)

                 from  n    params  module                                  arguments
  0                -1  1       928  models.common.Conv                      [3, 32, 3, 2, None, 1, LeakyReLU(negative_slope=0.1)]
  1                -1  1     18560  models.common.Conv                      [32, 64, 3, 2, None, 1, LeakyReLU(negative_slope=0.1)]
  2                -1  1      2112  models.common.Conv                      [64, 32, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
  3                -2  1      2112  models.common.Conv                      [64, 32, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
  4                -1  1      9280  models.common.Conv                      [32, 32, 3, 1, None, 1, LeakyReLU(negative_slope=0.1)]
  5                -1  1      9280  models.common.Conv                      [32, 32, 3, 1, None, 1, LeakyReLU(negative_slope=0.1)]
  6  [-1, -2, -3, -4]  1         0  models.common.Concat                    [1]
  7                -1  1      8320  models.common.Conv                      [128, 64, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
  8                -1  1         0  models.common.MP                        []
  9                -1  1      4224  models.common.Conv                      [64, 64, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 10                -2  1      4224  models.common.Conv                      [64, 64, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 11                -1  1     36992  models.common.Conv                      [64, 64, 3, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 12                -1  1     36992  models.common.Conv                      [64, 64, 3, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 13  [-1, -2, -3, -4]  1         0  models.common.Concat                    [1]
 14                -1  1     33024  models.common.Conv                      [256, 128, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 15                -1  1         0  models.common.MP                        []
 16                -1  1     16640  models.common.Conv                      [128, 128, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 17                -2  1     16640  models.common.Conv                      [128, 128, 1, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 18                -1  1    147712  models.common.Conv                      [128, 128, 3, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 19                -1  1    147712  models.common.Conv                      [128, 128, 3, 1, None, 1, LeakyReLU(negative_slope=0.1)]
 20  [-1, -2, -3, -4]  1         0  models.common.Concat                    [1]
```

Figure 18: Loading YOLOv7 tiny model

```
      Epoch    gpu_mem       box       obj       cls     total    labels  img_size
      97/99      5.61G   0.01228  0.004168  0.003063   0.01951        43       640: 100% 110/110 [56:45<00:00, 30.96s/it]
                 Class      Images      Labels          P          R      mAP@.5  mAP@.5:.95:   0% 0/16 [00:00<?, ?it/s]/usr/local/lib/python3.8/dist-packages/t
return _VF.meshgrid(tensors, **kwargs)  # type: ignore[attr-defined]
                 Class      Images      Labels          P          R      mAP@.5  mAP@.5:.95: 100% 16/16 [00:28<00:00,  1.81s/it]
                   all        1995        2000      0.996       0.99       0.993       0.917

      Epoch    gpu_mem       box       obj       cls     total    labels  img_size
      98/99      9.57G   0.01153  0.004184  0.002623   0.01833        55       640: 100% 110/110 [05:37<00:00,  3.07s/it]
                 Class      Images      Labels          P          R      mAP@.5  mAP@.5:.95: 100% 16/16 [00:25<00:00,  1.59s/it]
                   all        1995        2000      0.995      0.992       0.993       0.916

      Epoch    gpu_mem       box       obj       cls     total    labels  img_size
      99/99      9.57G   0.01148  0.004179  0.003031   0.01869        55       640: 100% 110/110 [05:36<00:00,  3.06s/it]
                 Class      Images      Labels          P          R      mAP@.5  mAP@.5:.95: 100% 16/16 [00:27<00:00,  1.72s/it]
                   all        1995        2000      0.995      0.993       0.993       0.918
                  fake        1995         248      0.988      0.972       0.993       0.877
                 fifty        1995         250      0.998          1       0.996        0.95
                  five        1995         253      0.997      0.988       0.986       0.929
           fivehundred        1995         250      0.998          1       0.997       0.923
               hundred        1995         249      0.995          1       0.997       0.888
                   ten        1995         250      0.996          1       0.995        0.95
              thousand        1995         251      0.994      0.984       0.983       0.917
                 twenty        1995         249      0.992          1       0.995        0.91
```

Figure 19: Output from Training

RESULTS

Output result evaluation :

- confusion matrix
- loss graph
- precision curve
- recall curve
- mAP curve

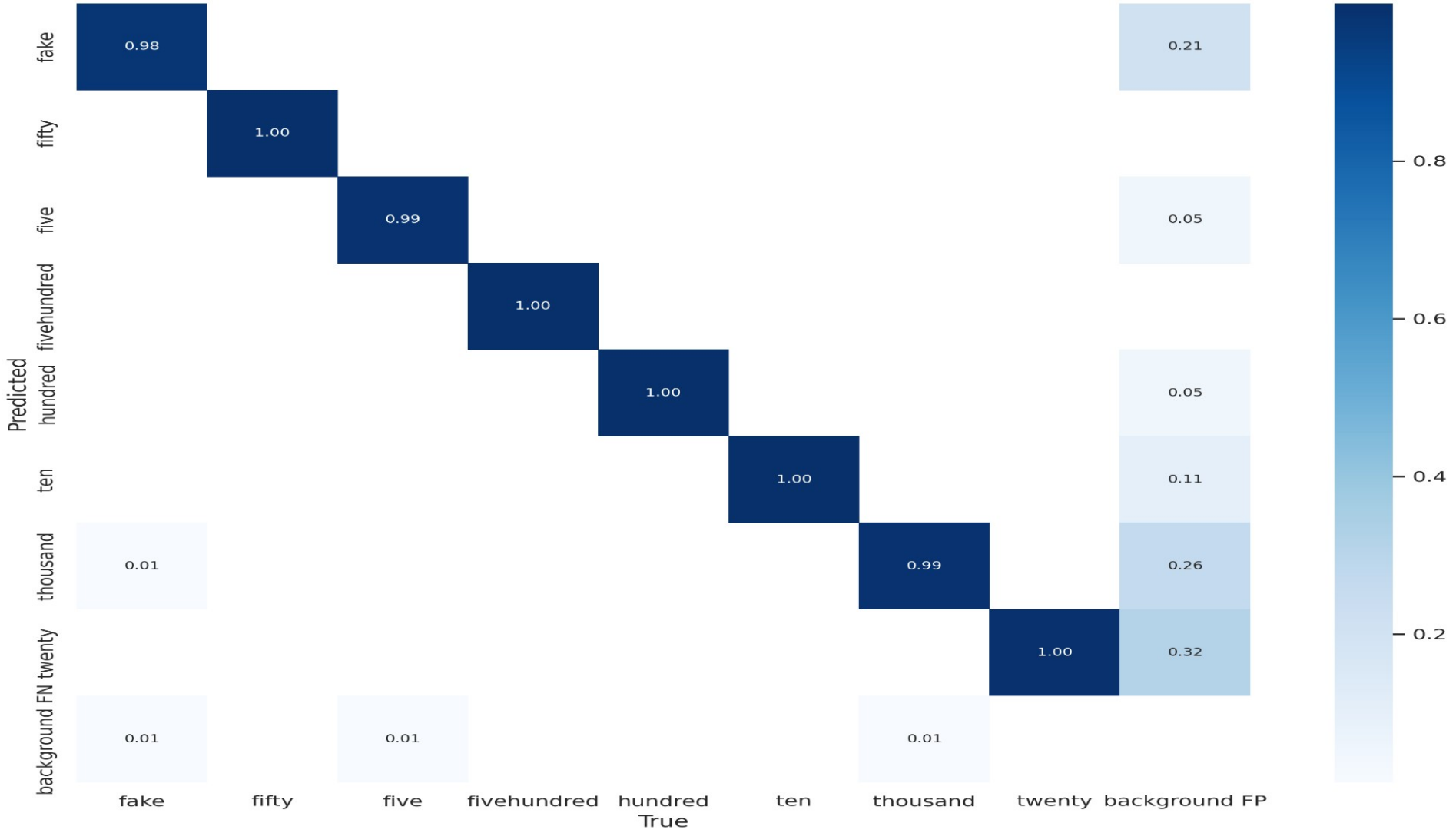


Figure 20: Confusion Matrix

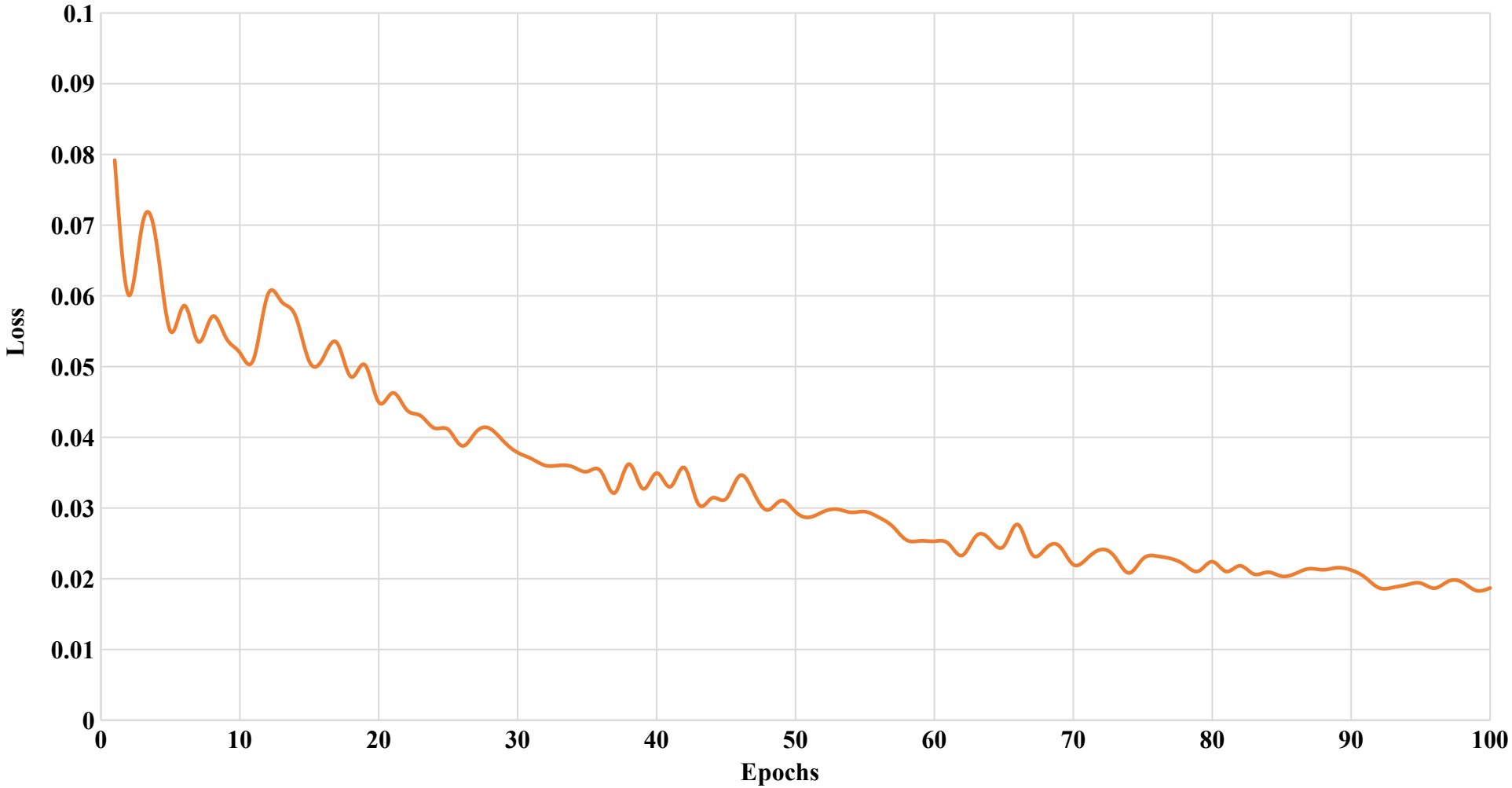


Figure 21: Loss Graph

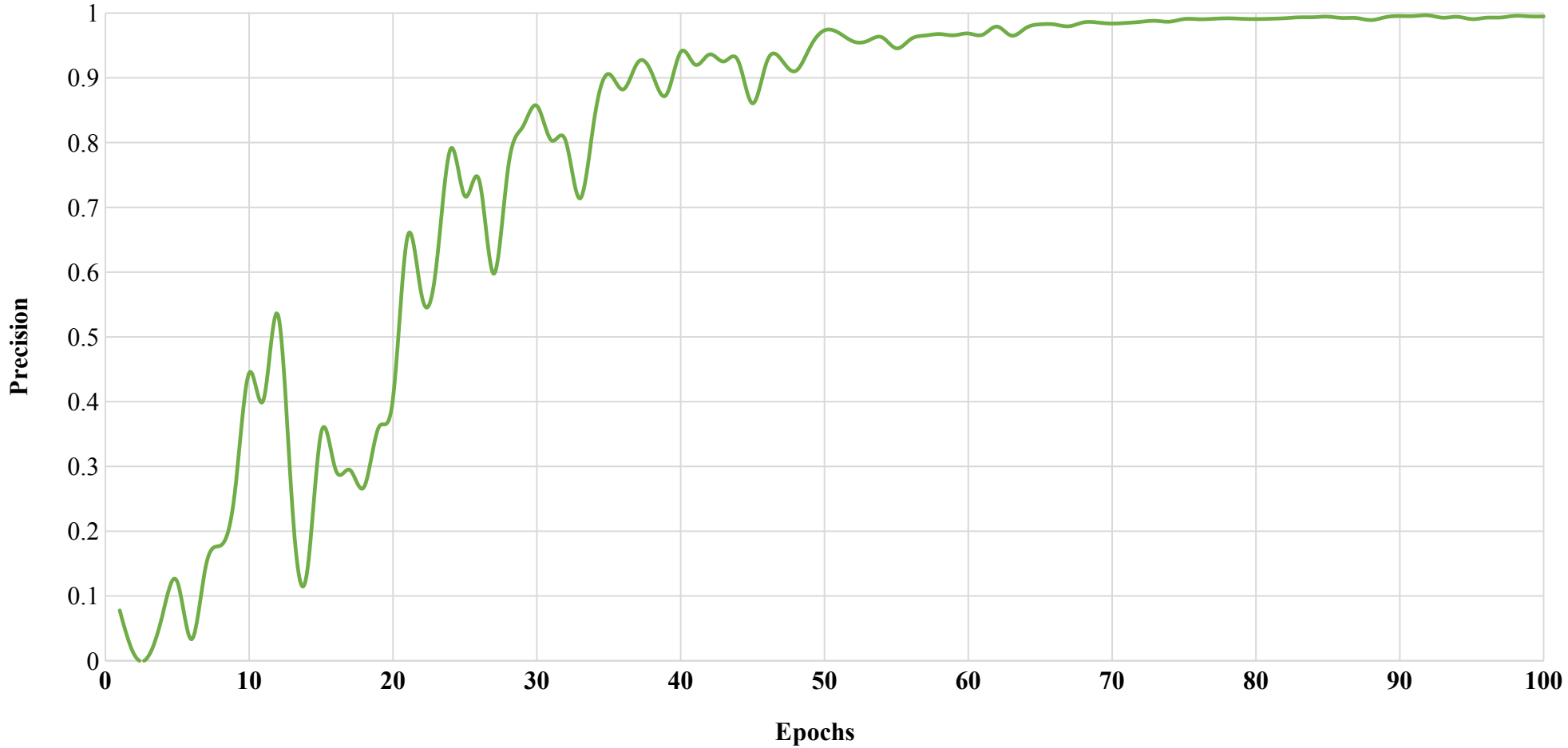


Figure 22: Precision Curve

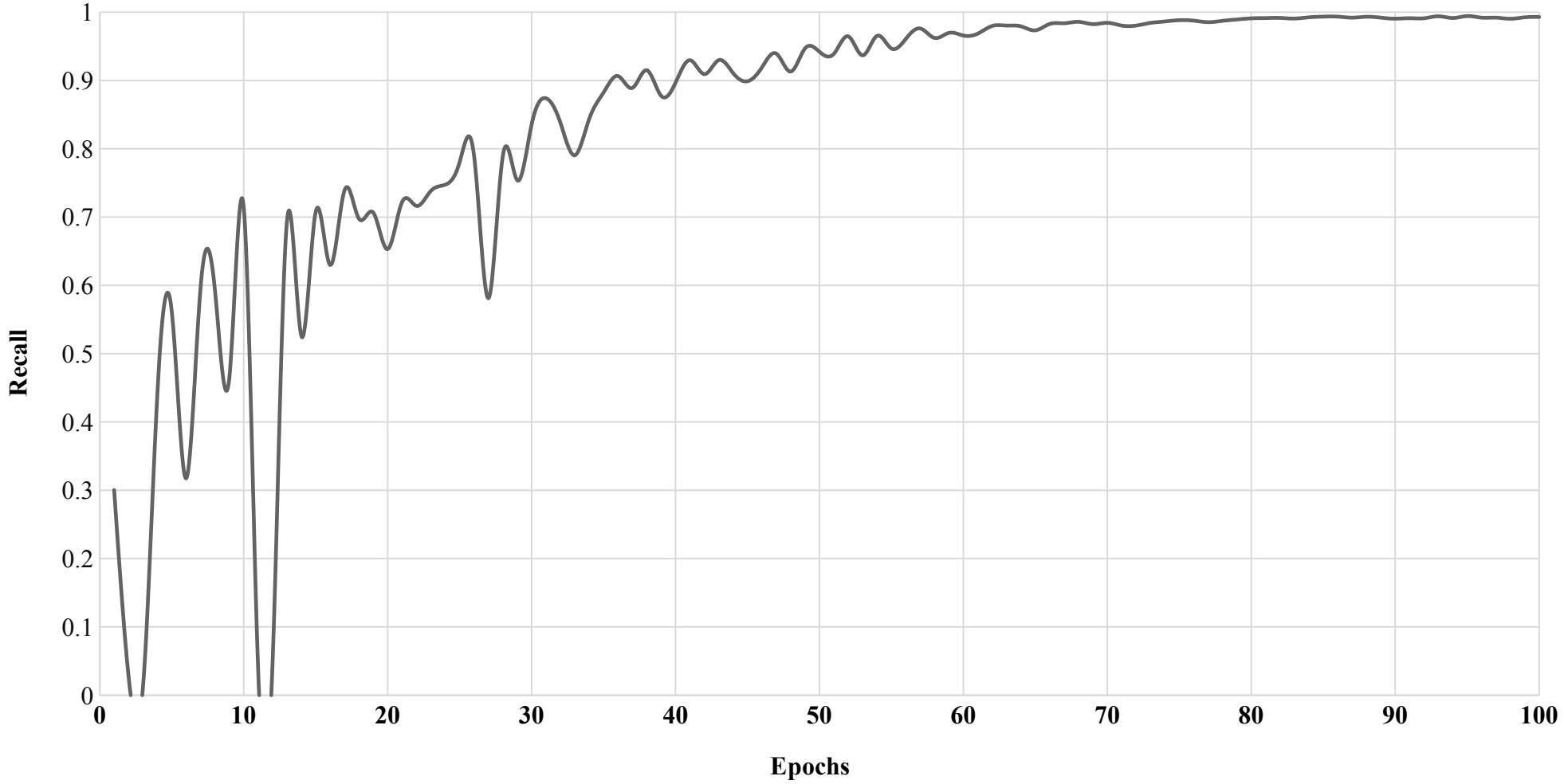


Figure 23: Recall Curve

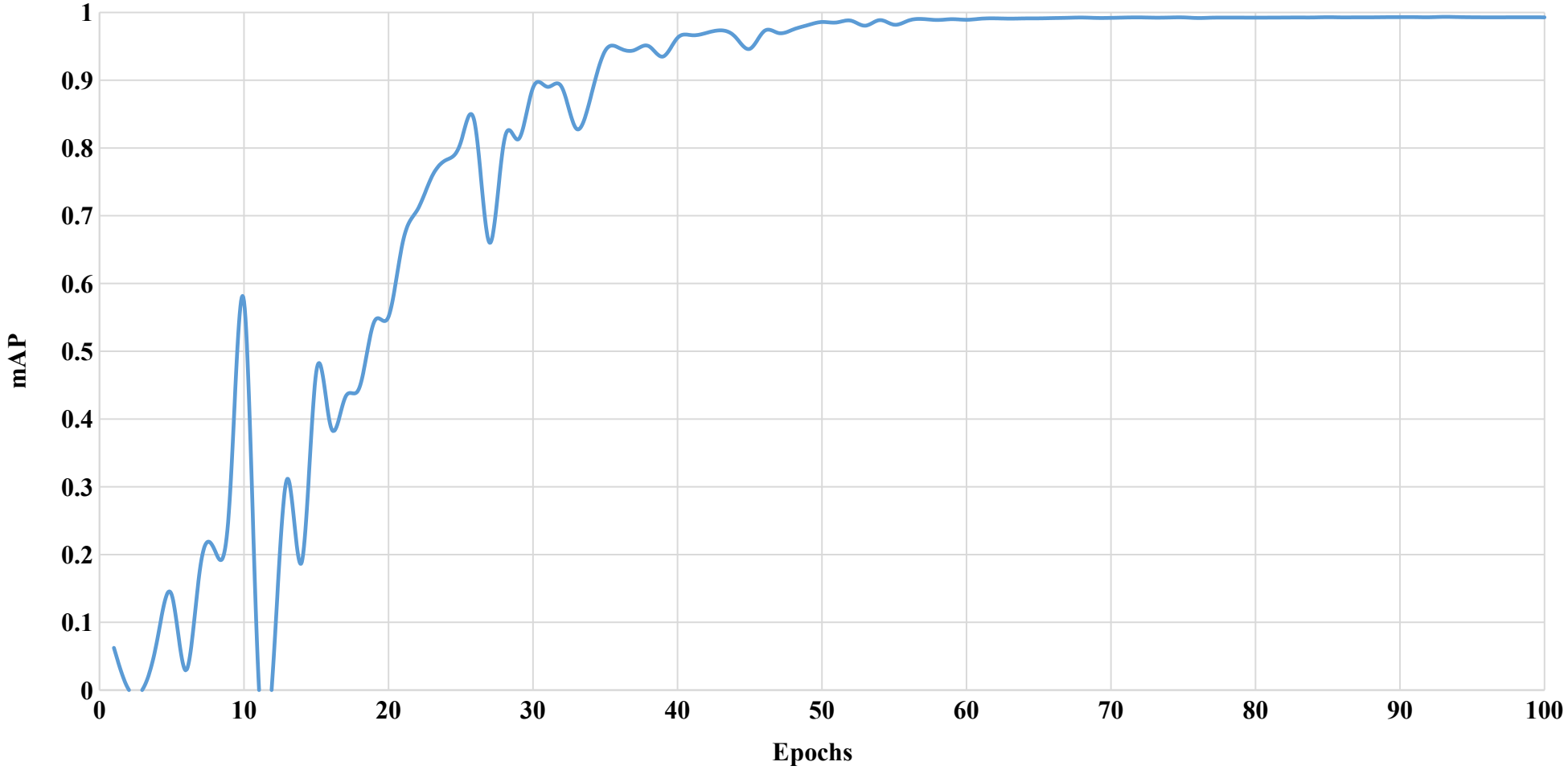


Figure 24: mAP Curve

```
test: Scanning 'data/test/labels.cache' images and labels... 1010 found, 0 missing, 1 empty, 0 corrupted: 100% 1010/1010 [00:00<?, ?it/s]
               Class     Images     Labels          P          R     mAP@.5  mAP@.5:.95: 100% 16/16 [00:17<00:00,  1.10s/it]
                 all       1010       1009      0.987      0.997      0.995       0.92
                fake       1010        128      0.969      0.979      0.989      0.887
               fifty       1010        125      0.992          1      0.996      0.946
                five       1010        125      0.981          1      0.996      0.932
          fivehundred       1010        125      0.989          1      0.997      0.929
             hundred       1010        127      0.994          1      0.997      0.881
                 ten       1010        125      0.993          1      0.996      0.944
            thousand       1010        127      0.991          1      0.996      0.921
              twenty       1010        127      0.984          1      0.996       0.92
Speed: 3.6/2.8/6.4 ms inference/NMS/total per 640x640 image at batch-size 64
```

Figure 25: Testing of Trained Model on Test images



Figure 27: Testing the Trained Model on Images



Figure 28: Testing of Trained model on Images

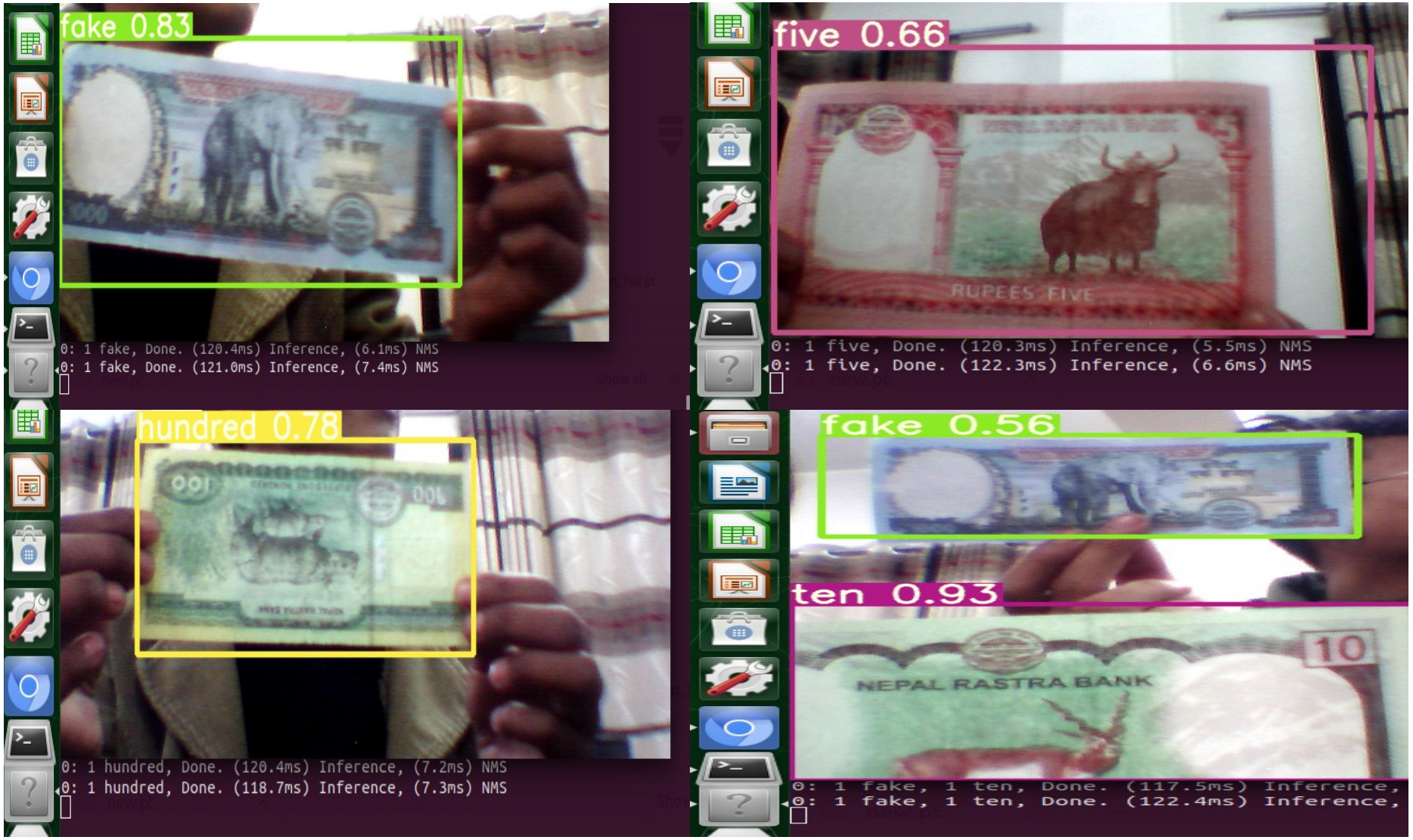


Figure 29: Real time Nepali currency detection in Jetson Nano

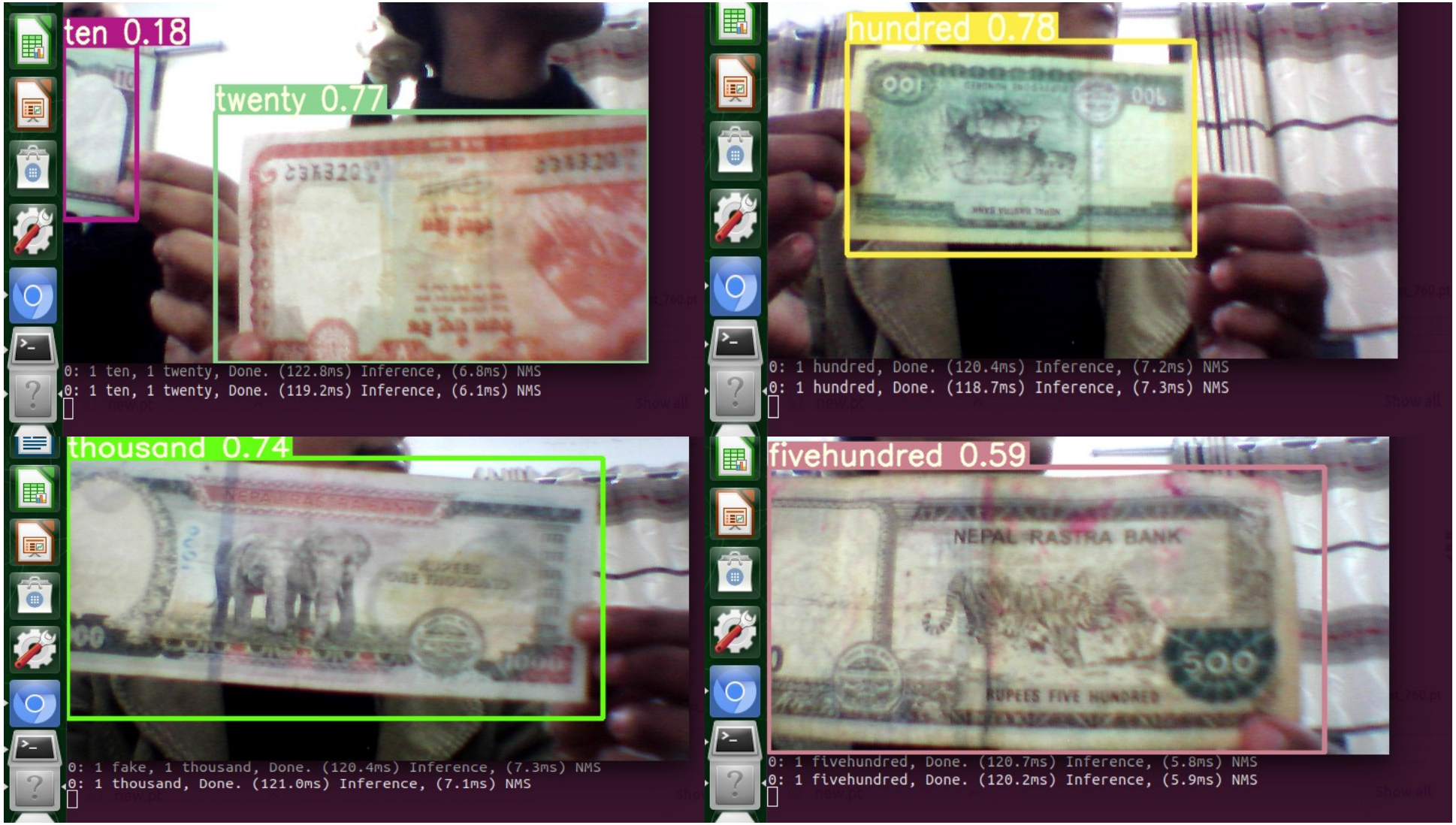


Figure 30: Real time Nepali currency detection in Jetson Nano

DISCUSSION

- The dataset was collected and trained in GPU service provided by Google Colaboratory with tiny YOLOv7.
- The training was time consuming but the service of Google Colaboratory was time and usage limited.
- Then trained model was tested on test images which resulted good precision and recall.

DISCUSSION

- Setting up the Jetson Nano we could perform denomination and counterfeit detection in real time.
- However, the resolution of camera and intensity of light affected the detection.

CONCLUSION

- The YOLOv7-tiny was successfully used in Nepali Paper Currency Recognition which is deep learning approach. The trained model can detect and recognize banknotes and counterfeit in images as well as in real time.
- The overall precision, recall and mAP for all classes was 0.995, 0.993 and 0.993 respectively.

REFERENCES

[1] S. Shrestha and R. Phoju, “International Journal of Advanced Engineering Fine Tuning YOLOv4 for Vehicle Detection,” vol. 03, no. 02, pp. 2586–7652, 2020, [Online]. Available: http://ictaes.org

[2] S. Indolia, A. K. Goswami, S. P. Mishra, and P. Asopa, “Conceptual Understanding of Convolutional Neural Network- A Deep Learning Approach,” Procedia Comput. Sci., vol. 132, pp. 679–688, 2018, doi: 10.1016/j.procs.2018.05.069.

[3] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” pp. 1–15, 2022, [Online]. Available: http://arxiv.org/abs/2207.02696

[4] S. Lamsal and A. Shakya, “Counterfeit Paper Banknote Identification Based on Color and Texture,” Proc. IOE Grad. Conf., pp. 160–168, 2015.

[5] V. Abburu, S. Gupta, S. R. Rimitha, M. Mulimani, and S. G. Koolagudi, “Currency recognition system using image processing,” 2017 10th Int. Conf. Contemp. Comput. IC3 2017, vol. 2018-Janua, pp. 1–6, doi: 10.1109/IC3.2017.8284300.

[6] M. Sarfraz, “An Intelligent Paper Currency Recognition System,” Procedia Comput. Sci., vol. 65, no. Iccmit, pp. 538–545, 2015, doi: 10.1016/j.procs.2015.09.128.

[7] A. Shah, K. Vora, and J. Mehta, “A Review Paper on Currency Recognition System,” Int. J. Comput. Appl., vol. 115, no. 20, pp. 1–4, 2015, doi: 10.5120/20264-2669.

[8] P Gayathri, “Texture Classification for Fake Indian Currency Detection,” Int. J. Eng. Res., vol. V9, no. 06, pp. 281–284, 2020, doi: 10.17577/ijertv9is060211.

[9] B. Padmaja, P. B. N. Shyam, H. G. Sagar, B. D. Nayak, and Mb. Rao, “Indian Currency Denomination Recognition and Fake Currency Identification,” J. Phys. Conf. Ser., vol. 2089, no. 1, 2021, doi: 10.1088/1742-6596/2089/1/012008.

[10] R. Wafia, R. Vrushabh, M. Chaitali, C. Dhiraj, and G. Kalyani, “Indian Currency Note Recognition System using YOLO v3 Methodology,” Int. Res. J. Eng. Technol., pp. 1349–1356, 2021, [Online]. Available: www.irjet.net

[11] S. Ashrit Dande, G. Reddy Uppunuri, and A. Singh Raghuvanshi, “YOLOv5 Based Web Application For Indian Currency Note Detection,” Int. Res. J. Eng. Technol., pp. 2801–2807, 2022, [Online]. Available: www.irjet.net

[12] G. Karimi, “Introduction to YOLO Algorithm for Object Detection,” 2021, [Online]. Available: https://www.section.io/engineering-education/introduction-to-yolo-algorithm-for-object-detection/

[13] Z. Keita, “YOLO Object Detection Explained,” 2022, [Online]. Available: https://www.datacamp.com/blog/yolo-object-detection-explained

[14] G. Nishad, “You Only Look Once(YOLO): Implementing YOLO in less than 30 lines of Python Code,” 2019, [Online]. Available: https://medium.com/analytics-vidhya/you-only-look-once-yolo-implementing-yolo-in-less-than-30-lines-of-python-code-97fb9835bfd2

[15] Kukil and S. Rath, “YOLOv7 Object Detection Paper Explanation & Inference,” 2022, [Online]. Available: https://learnopencv.com/yolov7-object-detection-paper-explanation-and-inference/

[16] K. E. Koech, “Confusion Matrix for Object Detection,” 2020, [Online]. Available: https://towardsdatascience.com/confusion-matrix-and-object-detection-f0cbcb634157

THANK YOU!!